Does Hardware Change Which Generative Model Is Efficient? Evaluating Diffusion and Flow-Matching Samplers Across Heterogeneous GPUs

Iason-Daniil Zakynthinos

Research proposal, Advanced Topics in Artificial Intelligence, 2026–2027

Thematic area: Generative AI for Images (secondary: Evaluation of AI Systems)

October 2026

Abstract

This project tests whether efficiency rankings of image generative models survive a change of hardware. Diffusion and flow-matching papers usually report efficiency as the number of function evaluations (NFE) or as latency on a single high-end GPU. Real deployments run on mixed hardware with different memory bandwidth, precision support and batch limits.

I will evaluate matched pretrained diffusion and flow-matching models (DiT and SiT, which share one architecture but differ in training objective) under several samplers, step counts and numerical precisions, on [3] GPU classes. For each GPU I will build speed–quality Pareto frontiers using FID, precision/recall, latency, memory and energy per image. I will then measure how well NFE-based rankings agree with hardware-measured rankings, and whether the two model families degrade differently when steps or precision are reduced.

The expected outcome is a reproducible measurement protocol and evidence on when NFE is, and is not, a trustworthy efficiency proxy.

1. Research question

The central question of this project is whether efficiency rankings of diffusion and flow-matching samplers stay the same across GPU classes, or whether the hardware changes which model and sampler counts as efficient. I break this into three sub-questions:

1. How well does NFE predict measured latency, throughput and energy on each GPU, and does that relationship depend on the model family or solver?

2. Do the speed–quality Pareto frontiers keep the same ordering of configurations across GPUs?
3. When steps or numerical precision are reduced, which family loses quality more slowly at the same measured cost?

My working hypotheses follow from how the two families sample. I expect NFE to track latency closely within a single GPU and solver, but to underestimate the cost of higher-order solvers and large batches on bandwidth-limited cards. If that holds, configurations near the Pareto frontier should change places between GPUs (I use Kendall's $\tau < 0.8$ between devices as the threshold), while clearly dominated configurations stay dominated. Finally, because flow-matching models are trained along straighter paths, I expect SiT to hold its quality better than DiT below roughly 20 steps, regardless of hardware.

The project only evaluates pretrained models at inference time. It does not involve training new models or building a serving system.

2. Motivation

Claims that one generative model is faster than another usually rest on NFE, or on timings from a single high-end GPU. That is a reasonable shortcut, but it assumes a function evaluation costs roughly the same everywhere, and there are good reasons to doubt this. Higher-order ODE solvers do extra work per step, and how much that overhead costs depends on the device. A transformer denoiser can be compute-bound on an A100 and memory-bandwidth-bound on a T4, so the same NFE turns into very different wall-clock times. Precision support varies too: bf16 gives large speedups on recent GPUs and little or nothing on older ones, and it is not obvious that diffusion and flow-matching models tolerate lower precision equally well.

If rankings do shift with hardware, conclusions drawn on one machine will not carry over to where models are actually deployed. Dehghani et al. (2022) made a similar point for general models, showing that FLOPs, parameter counts and measured speed can disagree. This project asks the same question for diffusion and flow-matching samplers, where NFE is the standard efficiency measure. It also responds to the course direction of comparing diffusion and flow-based methods at a fixed compute budget, by asking what that budget should mean once the hardware varies.

3. Related work

Denoising diffusion (Ho et al., 2020) established that iterative denoising can produce high-quality images, at the cost of hundreds of sampling steps. Flow matching (Lipman et al., 2023) and rectified flow (Liu et al., 2023) instead learn a velocity field along simpler,

nearly straight paths between noise and data, which in principle makes few-step sampling easier. Transition matching (Shaul et al., 2025) generalises these ideas further and is a possible extension if time allows.

On the architecture side, DiT (Peebles & Xie, 2023) replaced the U-Net with a transformer operating on latent patches and showed that sample quality improves steadily with compute. SiT (Ma et al., 2024) keeps the DiT backbone unchanged and trains it with an interpolant objective that includes flow matching. Since the two models share the same structure, parameter count and GFLOPs, differences between them can be attributed mainly to the training objective and sampler, which makes them a natural pair for this study.

A separate line of work reduces the number of steps diffusion models need. DDIM (Song et al., 2021), DPM-Solver (Lu et al., 2022) and the EDM sampler (Karras et al., 2022) bring sampling down to tens of steps, while progressive distillation (Salimans & Ho, 2022) and consistency models (Song et al., 2023) go as low as one to four. Nearly all of these papers report their gains in NFE.

Measuring quality carefully is a problem in its own right. FID (Heusel et al., 2017) is still the standard metric, but its finite-sample bias depends on the model being evaluated (Chong & Forsyth, 2020), and it is sensitive to low-level choices such as image resizing (Parmar et al., 2022). Improved precision and recall (Kynkäänniemi et al., 2019) help by separating fidelity from diversity. On the cost side, Dehghani et al. (2022) showed that common efficiency indicators can contradict each other, and Henderson et al. (2020) argued for systematic energy reporting in ML research.

What I have not found is a study that checks whether NFE-based rankings of diffusion and flow-matching samplers hold up across GPU classes and precisions. That is the gap this project addresses.

4. Methodology

The study is a controlled factorial evaluation: every model–sampler–steps–precision configuration is run on every GPU with the same images, seeds and software.

4.1 Models

I will use the official DiT-XL/2 (Peebles & Xie, 2023) and SiT-XL/2 (Ma et al., 2024) checkpoints for class-conditional ImageNet at 256×256. Both have the same backbone and parameter count, so differences between them isolate the training objective and sampler. If time allows, I will add one distilled few-step model as a reference point for the extreme low-step regime.

4.2 Experimental factors

Factor	Levels
Sampler (DiT)	DDPM ancestral (Ho et al., 2020), DDIM (Song et al., 2021), DPM-Solver (Lu et al., 2022)
Sampler (SiT)	Euler ODE, Heun ODE, Euler–Maruyama SDE
Sampling steps	8, 16, 32, 64, 128, 250
Precision	fp32, fp16, bf16 (where supported)
Batch size	1, 8, 32 (memory permitting)
GPU	[GPU A, e.g. T4], [GPU B, e.g. RTX 5080], [GPU C, e.g. A100]

Classifier-free guidance is fixed at one scale (e.g. 1.5) for all runs and counted in NFE, since it doubles the network evaluations per step.

4.3 Measurement protocol

1. Pin library versions (PyTorch, CUDA, cuDNN) and record driver and GPU clocks for every run.
2. Run warm-up iterations before timing; then time 10 repetitions with CUDA events and report median and interquartile range.
3. Record peak GPU memory and energy per image from NVML power sampling.
4. Keep compilation settings identical (e.g. `torch.compile` off for all runs, or on for all).
5. Use fixed seeds and class labels, so every configuration generates from the same noise and conditions.

4.4 Controls

Every FID is computed with the same number of images, the same resizing pipeline (clean-fid, following Parmar et al., 2022) and the same reference statistics. Before running any sweeps, I will check that the official sampling code reproduces the FID reported in each original paper. Image quality does not depend on the GPU, so it is measured once per configuration and precision, while cost is measured on every device. I will spot-check a few configurations across GPUs to confirm that numerical differences do not change the outputs in a meaningful way.

5. Data and resources

No new data is needed. The project uses the public DiT-XL/2 and SiT-XL/2 checkpoints, the ImageNet 256 reference statistics from the original papers, and standard tooling: PyTorch, the official DiT and SiT repositories, clean-fid for quality metrics and NVML (through pynvml) for power and memory. Generated samples need roughly 50–100 GB of temporary storage and are deleted once metrics are computed.

Most of the compute goes into quality evaluation rather than timing. Running about 108 configurations at 5,000 images each should take roughly 40–80 GPU-hours on an A100-class card, and this only has to happen once, on the fastest GPU available. Timing sweeps run on every device but need only a few batches per configuration, adding about 5–15 GPU-hours per GPU. If compute turns out to be tight, I can drop fp32 from the quality sweep, use 2,000 images for exploratory configurations while keeping 10,000 for those on the frontier, and reduce the step grid to 8, 16, 32, 64 and 250.

At the moment I have access to my own RTX 5080 and to free cloud GPUs such as Colab and Kaggle. I would like to ask whether any lab GPUs could be made available for the project, ideally a data-centre card such as an A100 or similar. This would cover most of the quality sweep and add a third, clearly different hardware class to the comparison. If lab hardware is not available, the fallback described in Section 7 applies.

6. Evaluation plan

For each research question I fix the metric, the analysis and a decision rule in advance, so that a negative result is as interpretable as a positive one.

6.1 Metrics

Type	Metric
Quality	FID (5k–10k samples, 3 seeds, 95% confidence intervals), precision and recall, Inception Score as a secondary check
Cost	NFE, median latency per image, throughput (images/s), peak memory (GB), energy per image (J)

6.2 Baselines

The reference diffusion setup is DiT with DDPM and DDIM sampling. The more important baseline, though, is the ranking of configurations by NFE alone, since that is the current practice this project puts to the test.

6.3 Analysis per research question

RQ	Analysis	Decision rule
RQ1	Regress measured latency and energy on NFE per GPU, sampler and batch size; report R^2 and residual patterns	NFE is an adequate proxy if $R^2 > 0.95$ within each GPU and residuals show no sampler-specific bias
RQ2	Build quality–latency and quality–energy frontiers per GPU; compute Kendall's τ between GPU rankings and between NFE and measured rankings	Rankings are hardware-dependent if $\tau < 0.8$ or any frontier configuration becomes dominated on another GPU
RQ3	Plot FID against measured cost as steps and precision fall; compare the slopes of DiT and SiT	One family degrades more gracefully if its FID increase is lower at matched cost, with non-overlapping confidence intervals

Thresholds are initial choices and will be justified or revised after pilot runs.

6.4 Interpreting outcomes

Either outcome is useful. If rankings change across GPUs, NFE-only comparisons need to be supplemented with measured cost, and the results will show which settings drive the change. If rankings stay stable, that is evidence that NFE is a safe proxy within the tested range, and the protocol gives others a way to check the same for new models.

7. Risks and mitigations

The main risk is hardware access. If I can only reach one or two GPU classes, I will combine Colab or Kaggle with my own card and treat batch size and precision as additional hardware-like axes. FID noise at small sample sizes is close to certain, so I will report confidence intervals over three seeds, use 10,000 samples for frontier configurations and use precision and recall as a second view.

If the quality sweep needs more compute than I have, I will apply the cuts described in Section 5, prioritising the step grid over the precision grid. Differences between the DiT and SiT codebases could also confound the comparison, which is why reproducing the

published FID for both comes first. Finally, timings on shared or throttled GPUs can be noisy, so I will record clocks and temperatures, report medians and rerun outliers.

8. Expected contributions

The project should produce three things. The first is evidence on whether NFE-based efficiency rankings of diffusion and flow-matching samplers survive a change of hardware and precision. The second is a controlled DiT-versus-SiT comparison showing how the training objective affects robustness to fewer steps and lower precision. The third is a released set of quality and cost measurements, with code, so that new samplers can be checked on the same hardware.

References

- Chong, M. J., & Forsyth, D. (2020). Effectively Unbiased FID and Inception Score and Where to Find Them. CVPR.
- Dehghani, M., et al. (2022). The Efficiency Misnomer. ICLR.
- Henderson, P., et al. (2020). Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning. JMLR, 21(248).
- Heusel, M., et al. (2017). GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. NeurIPS.
- Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. NeurIPS.
- Karras, T., et al. (2022). Elucidating the Design Space of Diffusion-Based Generative Models. NeurIPS.
- Kynkäänniemi, T., et al. (2019). Improved Precision and Recall Metric for Assessing Generative Models. NeurIPS.
- Lipman, Y., et al. (2023). Flow Matching for Generative Modeling. ICLR.
- Liu, X., Gong, C., & Liu, Q. (2023). Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. ICLR.
- Lu, C., et al. (2022). DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps. NeurIPS.
- Ma, N., et al. (2024). SiT: Exploring Flow and Diffusion-based Generative Models with Scalable Interpolant Transformers. ECCV.
- Parmar, G., Zhang, R., & Zhu, J.-Y. (2022). On Aliased Resizing and Surprising Subtleties in GAN Evaluation. CVPR.
- Peebles, W., & Xie, S. (2023). Scalable Diffusion Models with Transformers. ICCV.
- Salimans, T., & Ho, J. (2022). Progressive Distillation for Fast Sampling of Diffusion Models. ICLR.

- Shaul, N., Singer, U., Gat, I., & Lipman, Y. (2025). Transition Matching: Scalable and Flexible Generative Modeling. NeurIPS.
- Song, J., Meng, C., & Ermon, S. (2021). Denoising Diffusion Implicit Models. ICLR.
- Song, Y., et al. (2023). Consistency Models. ICML.